

MILO – Part: Implementation of interaction between IM948 and ArmPi robotic arm

Linkun Zhang, DC327216^a

^aElectrical and computer engineering apartment, University of Macau

Abstract—This report is part of the final report of the Project Design (MILO) group of ECEN3025 course in the 24/25 academic year, which is about realizing a human-machine interactive robot arm through a 9-axis sensor. This report starts with describing how to use Kalman filtering to achieve smooth and stable data reporting by IM948, and then describes how to use the ESP32 development board to remotely connect IM948 to the host computer, and how to use Kalman filtering on the host computer to process the original data of IM948, get rotation angle and integrate to obtain the corresponding displacement data. And then use QT6 to make the corresponding displacement and rotation animation. Later on is the communication between ESP32 and ArmPi, 6-axis data control of ArmPi servo motor movement, and simple inverse kinematics processing.

Keywords—IM948, ESP32, ArmPi, Inertial Navigation, Kalman Filter, Accelerometer, Inverse Kinematics, Robotic Arm, Human-computer Interaction, QT6 Interface Design

Contents

1	Introduction	1
2	Preparation	1
2.1	Software	1
2.2	Hardware	1
3	Architecture	1
4	Hardware Connection	2
5	Kalman Filter	2
6	IMU and ADC Data Acquisition and Transmission Process	3
7	Host PC IMU TCP Receiver	3
8	Real-Time IMU Visualization and MJPEG Streaming Interface	3
9	Position and Angle Tracking from IMU	4
10	Processed IMU Data transmitter to ArmPi with GUI Control	4
10.1	System Components and Workflow	4
11	ZMQ Remote ArmPi Position Inverse Kinematics Control System	5
12	Contact me	5

1. Introduction

In recent years, the integration of low-cost inertial measurement units (IMUs) with robotic systems has opened up new possibilities for real-time, intuitive, and wireless control of robotic arms. This project presents the design and implementation of an IM948-based robotic arm control system that uses real-time sensor data to perform inverse kinematics (IK) actuation and 3-axis Euler angle mapping of 3-axis robotic arm motor angle relationships. The system is built on a modular architecture and includes a MEMS module (IM948) an ESP32 microcontroller for data acquisition, Wi-Fi-based JSON format data transmission, and a Raspberry Pi as a control terminal.

IMU data represents three-axis direction vectors, transmitted via TCP or UDP protocols, and processed on the Raspberry Pi to calculate the target joint angles via an inverse kinematics algorithm. To ensure accurate execution of the motion, data normalization, offset correction, and filtering techniques are used. In addition, the control framework includes a graphical user interface for real-time monitoring and manual reset functions, which enhances user interactivity and operational safety.

The project focuses on communication flexibility (supporting Bluetooth and Wi-Fi), robustness of data parsing and synchronization, and scalability of real-time visualization and multi-modal control. The final system shows extremely high response speed and stability, suitable for scenarios such as education, scientific research, and human-computer interaction.

2. Preparation

2.1. Software

- Python IDE: PyCharm®
- ESP32 C++ IDE: Arduino IDE
- Language: Python (85%), C++ (10%), .bash (5%)
- ArmPi OS: Ubuntu, ROS2
- Host PC OS: Windows®11

2.2. Hardware

- Robotic Arm: ArmPi
- ESP32 C3 Mini
- 600mAh, 3.7V Li-ion Battery
- 3.3V Voltage Regulator Module
- MEMS: IM948
- 350Ω, 15cm Strain Gauge
- Bending Sensor Module: BF350
- Gloves
- 4.2V Li-ion Battery Charger Module
- Switch

3. Architecture

Figure 1 shows the overall architecture and data flow of the IMU-driven robotic arm motion control system developed in this project. First, the IM948 inertial measurement unit (IMU) sends real-time motion data to the ESP32 microcontroller through UART serial communication. ESP32 then sends the data to the backend server through the TCP protocol for further processing.

After receiving the data, the backend server first eliminates the noise and error in the measurement through the Kalman filter to obtain stable and reliable measurement data. The filtered data enters two processing channels respectively:

1. Displacement calculation channel: calculates the spatial displacement of the object measured by the IMU.
2. Euler Angle calculation channel: converts the raw data into Euler angle information represented by the spatial posture.

These two parts of data are transmitted to the 3D animation module through the ZMQ message queue to generate motion visualization effects in real time. The 3D animation module then sends the processed animation information back to the center server (Center Server) through the TCP protocol to achieve data visualization and real-time monitoring.

As the core node of the entire system, the central server collects all data streams through the ZMQ communication interface and selects different robot control strategies according to the currently set operation mode (Mode):

1. When Mode=1, the system executes Inverse Solution to calculate the joint angle of the robot target point;

- When Mode $\neq$ 1, Angle Mapping is executed to directly map the current posture to the robot action command.

Finally, the control commands of both modes are sent to the ArmPi robot to perform the corresponding actions, completing the real-time motion following and control tasks.

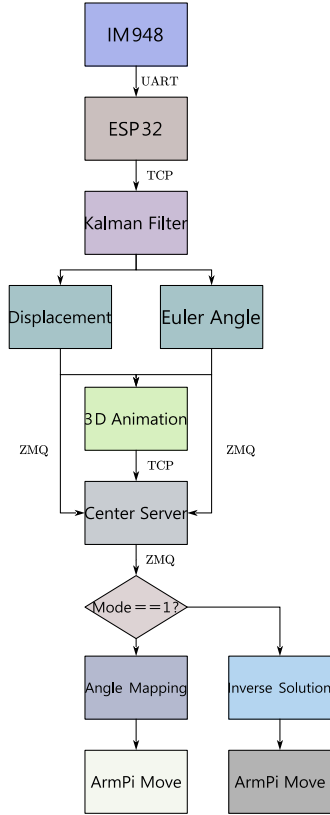


Figure 1. The whole system structure

4. Hardware Connection

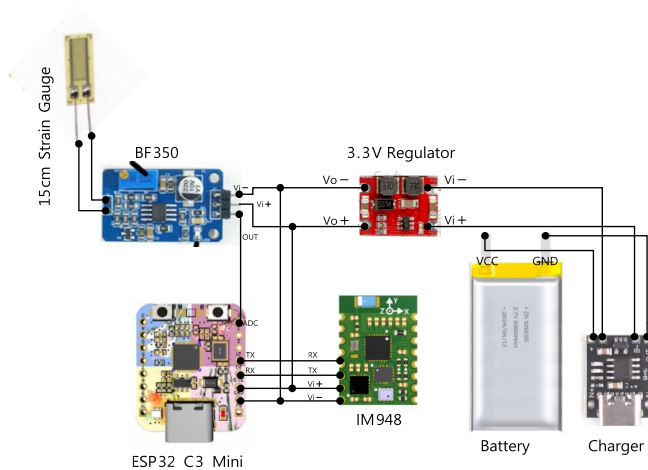


Figure 2. The circuit scheme

Figure 2 shows the overall circuit design used in this project. The system integrates a strain gauge sensor, an IMU (IM948), and an ESP32-C3 microcontroller, all powered by a rechargeable lithium battery regulated to 3.3V.

The 15 cm strain gauge is connected to the BF350 module, which serves as the signal conditioning unit. The output of the BF350 is

connected to the ADC (Analog-to-Digital Converter) pin of the ESP32-C3 Mini, allowing real-time monitoring of mechanical deformation data.

The IM948 inertial measurement unit communicates with the ESP32 via UART, with the TX pin of the IM948 connected to the RX pin of the ESP32, and vice versa. Both the BF350 and IM948 modules, as well as the ESP32 itself, are powered through a 3.3 V regulator. The regulator takes input from a 3.7 V lithium battery and provides a stable 3.3 V output to all connected modules.

To manage power, a lithium battery charging module is included in the design. The charger is connected to the battery, providing charging via USB, while simultaneously powering the 3.3 V regulator. This setup ensures stable operation of all modules during both battery and charging conditions.



Figure 3. The connected hardware stuck to the glove

5. Kalman Filter

State Definition

The filter state vector $\mathbf{x} \in \mathbb{R}^{10}$ consists of:

- Quaternion orientation $\mathbf{q} = [q_0, q_1, q_2, q_3]^T$
- Angular velocity $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$
- Accelerometer measurement in IMU body frame $\mathbf{a}_{imu} = [a_x, a_y, a_z]^T$

$$\mathbf{x} = \begin{bmatrix} \mathbf{q} \\ \boldsymbol{\omega} \\ \mathbf{a}_{imu} \end{bmatrix} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \\ \omega_x \\ \omega_y \\ \omega_z \\ a_x \\ a_y \\ a_z \end{bmatrix}$$

State Transition Function f_x

The quaternion derivative is given by:

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \boldsymbol{\omega}_q$$

where $\boldsymbol{\omega}_q = [0, \omega_x, \omega_y, \omega_z]^T$ and $\otimes$ denotes quaternion multiplication.

With Euler integration:

$$\mathbf{q}_{k+1} = \text{normalize} \left(\mathbf{q}_k + \frac{1}{2} \cdot (\mathbf{q}_k \otimes \boldsymbol{\omega}_q) \cdot \Delta t \right)$$

Thus, the full state update becomes:

$$\mathbf{f}_x(\mathbf{x}_k, \Delta t) = \begin{bmatrix} \mathbf{q}_{k+1} \\ \boldsymbol{\omega}_k \\ \mathbf{a}_{\text{imu},k} \end{bmatrix}$$

Measurement Function h_x

The measurement model directly observes the full state:

$$\mathbf{z}_k = \mathbf{h}_x(\mathbf{x}_k) = \mathbf{x}_k$$

Rotating Acceleration to World Frame

The IMU acceleration vector is rotated from the body frame to the world frame using the quaternion rotation:

$$\mathbf{a}_{\text{world}} = \mathbf{R}(\mathbf{q}) \cdot \mathbf{a}_{\text{imu}}$$

Where $\mathbf{R}(\mathbf{q})$ is the rotation matrix derived from the quaternion:

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} 1 - 2(q_2^2 + q_3^2) & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & 1 - 2(q_1^2 + q_3^2) & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & 1 - 2(q_1^2 + q_2^2) \end{bmatrix}$$

Velocity and Position Integration

If the norm of the rotated acceleration exceeds a threshold ϵ (e.g., 0.2), velocity and position are updated via numerical integration:

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{a}_{\text{world}} \cdot \Delta t$$

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_{k+1} \cdot \Delta t$$

UKF Filtering Process

Prediction Step

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{f}_x(\hat{\mathbf{x}}_{k-1}, \Delta t)$$

Update Step

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \hat{\mathbf{z}}_{k|k-1})$$

where the Kalman gain $\mathbf{K}_k$ is computed via the Unscented Transform from the sigma points.

6. IMU and ADC Data Acquisition and Transmission Process

This section describes the data flow of the ESP32C3 program, which receives IMU data over UART, appends ADC readings, and transmits the final packet via TCP to a host PC.

- WiFi Connection:** The ESP32C3 connects to a specified WiFi access point. Upon successful connection, the ESP32 obtains a local IP address and prepares to establish a TCP connection.
- IMU Configuration:** The ESP32 sends initialization command packets to the IM948 module via UART (at 115200 baud), setting its sampling rate and output format. These packets follow a specific frame structure starting with byte 0x11, 0x49 and ending with 0x4D, including address, payload length, payload data, and checksum.
- UART Data Parsing:** An asynchronous UART event task reads incoming bytes from the IMU and reconstructs valid data packets by detecting frame boundaries and verifying the checksum. When a complete packet is received, a global flag `packet_ready` is set to true.
- ADC Sampling:** When a complete IMU packet is ready, the ESP32 performs a 12-bit ADC read from pin GPIO34 (ADC1_CH6). The raw ADC value is split into two bytes (high and low) and inserted into the payload before the checksum. This raw ADC data reflects the deformation degree of the strain gauge.

5. Packet Modification and Update:

- The original payload length is incremented by 2 to accommodate the ADC bytes.
 - The new checksum is recomputed over the updated payload.
 - The total packet length is adjusted accordingly.
- TCP Transmission:** A TCP client task maintains a connection to the server at 192.168.1.XX:9999. When a packet is ready, it sends the modified IMU + ADC packet via TCP. The system enforces a minimum frame interval of 10 ms to limit the transmission rate.
 - Reconnection Logic:** If the TCP connection is lost or an error occurs during transmission, the client task will automatically attempt to reconnect after a short delay.

Finally the payload frame looks like:

```
[0x11] [0x49] [Control Flags (2 bytes)] [Timestamp]
[AccX] [AccY] [AccZ] ... [ADC (2 bytes)] [0x4D]
```

The overall system ensures reliable delivery of synchronized IMU and ADC data by preserving the original packet structure and dynamically updating the frame with new sensor data in real-time.

7. Host PC IMU TCP Receiver

The `IMUTCPReceiver` class implements an asynchronous TCP server that receives binary IMU data from an ESP32 client, parses the payload, and shares the latest values.

- TCP Server:** Listens on port 9999 and handles incoming sensor packets from ESP32 devices.
- Packet Format:** Each packet starts with 0x49, ends with 0x4D, and includes a payload with a 2-byte control flag indicating which sensors are active.
- Data Parsing:** Supported fields include acceleration, gyroscope, magnetometer, quaternion, temperature, pressure, orientation angles, offsets, activity, and ADC.
- ADC Support:** The final two bytes represent a 12-bit ESP32 ADC reading, converted to voltage and added to the parsed data.
- Shared Access:** Parsed results are stored in a shared dictionary and written to `imu_data_cache.json` for backup access.
- Fallback Mode:** If TCP binding fails, the instance switches to file-polling mode and reads from the cache.

Access to individual sensor values is provided through getter methods such as `get_acceleration()`, `get_gyro()`, and `get_adc_esp32()`.

8. Real-Time IMU Visualization and MJPEG Streaming Interface

This Python script implements an integrated real-time visualization system for IMU data received over TCP. It features a PyQt-based GUI and an HTTP MJPEG stream for remote viewing.

Architecture Overview

- TCP Receiver:** Uses `IMUTCPReceiver` to collect IMU quaternion, gyroscope, and accelerometer data from an ESP32-based sensor over TCP.
- Filtering:** The `IMUFilter` class applies an Unscented Kalman Filter (UKF) to smooth sensor data and estimate orientation and linear acceleration in the world frame.
- GUI Components (PyQt6):**
 - 2D Plots:** Raw and filtered gyroscope/accelerometer signals over time.
 - 3D Trajectory:** Plots estimated position in 3D space over time.

- **3D Cube Display:** Visualizes current orientation via a rotated cube using rotation matrix R derived from the quaternion.
- **Text Readouts:** Current acceleration vector and Euler angles (roll, pitch, yaw).
- **Start/Stop Buttons:** Control real-time plotting.

- **MJPEG Streaming:**

- Captures rendered 2D, 3D, and cube plots.
- Concatenates them horizontally into a single JPEG frame.
- Streams the frames via a multithreaded MJPEG HTTP server on port 7777.
- Accessible in browsers or video clients at `http://<host>:7777/`.

- **Execution Flow:**

1. Launches the TCP receiver in a separate thread.
2. Initializes and displays the PyQt GUI.
3. Starts the MJPEG server in the background.
4. Updates the visualization every 16 ms using a QTimer.

This system provides a robust framework for real-time motion visualization and streaming, useful for robotics, IMU testing, or gesture-based control applications.

9. Position and Angle Tracking from IMU

The `IMUPosition` class is responsible for tracking cumulative position, orientation, and ADC values from an ESP32-based IMU system over TCP. It connects to a shared IMU TCP receiver `IMUTCPReceiver` class, extracts displacement and orientation offset data, and applies correction logic to accumulate a real-time position estimate.

Data Processing Pipeline

1. **IMU Receiver Initialization:** An instance of `IMUTCPReceiver` is created to asynchronously receive incoming IMU data via TCP.
2. **Real-Time Data Collection:** The method `update_position()` retrieves offset values ($\Delta x, \Delta y, \Delta z$) from the IMU at fixed intervals (every 0.1 seconds).
3. **Axis Compensation:** Small correction factors are applied:
 - Z-axis: +10% for negative, –5% for positive.
 - X and Y: +10% for positive.
4. **Motion Detection:** If offsets are near zero, the system is assumed stationary. During motion, the dominant axis is determined and filtered to suppress non-dominant directions.
5. **Position Accumulation:** At the end of a motion phase, the last dynamic offset is clipped to the range $[-0.3, +0.3]$ and added to the cumulative position vector $\mathbf{p} = [p_x, p_y, p_z]$.
6. **Trajectory Logging:** Time-stamped position snapshots are stored and trimmed to retain only the most recent 10 seconds.
7. **ESP32 ADC Tracking:** The method `get_current_adc()` computes the voltage difference from an initial baseline, allowing drift detection or physical displacement measurement.
8. **Angle Correction:** For startup calibration, the first 2 seconds of raw angle data are used to determine whether axis values cross the 180° boundary. Corrective offsets are then applied accordingly.

Dominant Axis Detection and Filtering During Motion

When the IMU offset readings indicate non-zero movement, the system enters a dynamic state. To reduce noise and suppress unintended drift, the algorithm identifies a **dominant axis** of motion and selectively filters out the others.

1. **Threshold-Based Detection:**

- Let $\mathbf{r} = [x, y, z]$ be the current offset reading.

- Compute absolute values: $|x|, |y|, |z|$.
- Set a small threshold $\epsilon = 0.05$.
- The axis with the largest magnitude exceeding ϵ is considered dominant.

Formally:

$$\text{dominant axis} = \begin{cases} x & \text{if } |x| > \max(|y|, |z|) \text{ and } |x| > \epsilon \\ y & \text{if } |y| > \max(|x|, |z|) \text{ and } |y| > \epsilon \\ z & \text{if } |z| > \max(|x|, |y|) \text{ and } |z| > \epsilon \\ \text{all} & \text{otherwise} \end{cases}$$

2. **Axis Suppression:**

- If a single dominant axis is found, only its component is retained.
- The other two components are zeroed out: $r_j = 0$ for $j \neq \text{dominant}$.

3. **Peak Holding:**

- During continuous motion, the system updates the stored peak value along the dominant axis:

$$\text{last_dynamic}_{\text{dom}} = \begin{cases} \max(\text{last_dynamic}_{\text{dom}}, r_{\text{dom}}) & \text{if } r_{\text{dom}} > 0 \\ \min(\text{last_dynamic}_{\text{dom}}, r_{\text{dom}}) & \text{if } r_{\text{dom}} < 0 \end{cases}$$

- This prevents over-accumulation of small fluctuating values and ensures only meaningful movement is captured.

When the motion ends (i.e., when $\mathbf{r} \approx \mathbf{0}$), the stored dynamic offset is clipped to the range $[-0.3, +0.3]$ and added to the cumulative position vector.

Threading Architecture

- One background thread runs the asynchronous TCP receiver.
- A second thread executes the periodic `update_position()` loop.

User Interaction

- Pressing Enter clears the ADC baseline or resets the angle origin, depending on mode.
- The script prints current cumulative position, angle delta, or ADC change in real time.

10. Processed IMU Data transmitter to ArmPi with GUI Control

This module implements an interactive IMU data transmission interface using Python, combining real-time TCP-based IMU data acquisition, GUI controls via Tkinter, and asynchronous data transmission through a ZMQ client.

10.1. System Components and Workflow

1. **IMU Initialization:**

- An instance of `IMUPosition` is created to connect to the ESP32 IMU via TCP.
- Real-time displacement and angle data are computed from IMU offsets and corrected internally.

2. **Tkinter GUI:**

- The GUI provides buttons to start/stop transmission and reset both displacement and angle values.
- A radio switch allows toggling between two data modes:
 - Mode 0: Transmit cumulative position (x, y, z)
 - Mode 1: Transmit corrected angle $(\theta_x, \theta_y, \theta_z)$

3. Data Transmission Loop:

- Runs in an asynchronous loop (every 0.1 seconds).
- If `imu.sending` is True, the current data is collected, rounded, and sent to the ZMQ server using `TransmissionClient`.
- Each message is a list of the form:

$$[\text{mode}, x, y, z, \text{voltage}]$$

where `voltage` is the ESP32 ADC reading offset from baseline.

4. ACK Handling:

- The ZMQ client listens for acknowledgment messages labeled "ack" and prints them upon receipt.

5. Threading Model:

- The Tkinter GUI runs on a separate daemon thread.
- TCP reception and GUI interaction are decoupled from the asyncio-based message transmission loop.

This setup allows flexible, real-time IMU data capture and user-controlled transmission, suitable for interactive robotic control or wireless motion tracking.

ZMQ Remote ArmPi Servo Angle Control System

This script implements a ZMQ-based asynchronous control loop that receives processed IMU data over a network and adjusts servo positions accordingly in real time.

System Components

- **ZmqRemoteController:** Listens to remote IMU control signals over ZMQ at address `tcp://192.168.1.XX:5556`. The received data has the format:

$$[\text{prefix}, x, y, z, \text{volt}]$$

- **ServoController:** Controls six servo motors via the Hiwonder `BusServo` API. The initial positions are defined and dynamically adjusted based on incoming IMU readings.

Control Logic

1. When `prefix == 1`, the script enters **angle control mode**:

- Servo 1 position is updated based on `volt`:

$$p_1 = 100 + \frac{\text{volt}}{7}$$

- Servo 2 and 3 positions are updated proportionally to `x` and `y`:

$$p_2 = 500 + 3x, \quad p_3 = 268 + y$$

- Servo 4 mirrors Servo 3:

$$p_4 = 634 - y$$

- Servo 6 is updated based on `z`:

$$p_6 = 500 + 3z$$

2. The updated position array is passed to the servo controller to execute the movement.
3. The system sleeps for 0.1 seconds between updates to maintain a consistent frame rate.

This setup enables smooth and responsive remote control of servo-driven mechanisms based on IMU orientation and voltage data transmitted via a wireless network.

11. ZMQ Remote ArmPi Position Inverse Kinematics Control System

The `IMUActionExecutor` class enables real-time control of a robotic arm using inverse kinematics, driven by 3D positional data streamed from an IMU-based remote controller.

System Workflow

1. Input Handling:

- Receives data from a remote controller in the format:

$$[\text{prefix}, x, y, z, \text{volt}]$$

- Only executes position commands when `prefix == 0`.

2. Coordinate Mapping:

The raw IMU values (`x`, `y`, `z`) are scaled and offset to produce spatial coordinates:

$$x' = 80 \cdot x$$

$$y' = 30 \cdot y + 18$$

$$z' = 30 \cdot z + 18$$

These coordinates form the target position for the robotic arm.

3. Redundancy Check:

To reduce unnecessary motion, the executor compares the new coordinate with the previous one. If unchanged, no action is taken.

4. Inverse Kinematics Execution:

- A pitch angle constraint is applied: Target angle $\alpha = 0^\circ$, within a range $[-90^\circ, 0^\circ]$.
- The arm's inverse kinematics function `setPitchRangeMoving()` is called with the target coordinate and pitch constraint.
- If the motion is valid, it returns the computed servo positions and movement time, which are then printed.

12. Contact me

✉ Email: dc32721@um.deu.mo